Flag in Flame

⚠️ **Security Warning**: Work in an isolated environment (virtual machine or container). Never open unknown files on your primary machine.

📋 Challenge Description

The SOC team discovered a suspiciously large log file after a recent breach. When they opened it, they found an enormous block of encoded text instead of typical logs. Could there be something hidden within? Your mission is to inspect the resulting file and reveal the real purpose of it. The team is relying on your skills to uncover any concealed information within this unusual log.

💡 Hints

- Use `base64` to decode the data and generate the image file

🧰 Tools Used

- **file**: Utility to identify file types
- **strings**: Command-line tool to extract readable ASCII sequences from files
- **grep**: Text search utility for filtering output
- **exiftool**: Metadata inspection tool
- **base64**: Encoding/decoding utility for Base64 data
- **xxd**: Hexadecimal viewer and editor for converting hex to text
- **cat**: File reading utility

🔍 Complete Solution

Step 1: Download and Prepare

Download the log file named `logs.txt` from the challenge page.

Step 2: Basic File Inspection

First, verify the file type and general information:

```
file logs.txt
```

Expected output:

```
logs.txt: ASCII text, with very long lines (65536), with no line terminators
```

Step 3: Search for Visible Text

The flag may be present as plain text. Try extracting all character strings:

```
strings logs.txt | grep -i "picoCTF"
```

⚠️ *If no results appear, proceed to the next step (decoding).*

Step 4: Inspect Metadata

Use `exiftool` to examine the file metadata:

```
exiftool logs.txt
```

Complete output:

```
ExifTool Version Number         : 13.50
File Name                       : logs.txt
Directory                       : .
File Size                       : 1592 kB
File Modification Date/Time     : 2025:10:01 01:28:40+02:00
File Access Date/Time           : 2026:05:05 11:38:53+02:00
File Inode Change Date/Time     : 2026:05:05 11:38:49+02:00
File Permissions                : -rw-r--r--
File Type                       : TXT
File Type Extension             : txt
MIME Type                       : text/plain
MIME Encoding                   : us-ascii
Newlines                        : (none)
Line Count                      : 1
Word Count                      : 1
```

Nothing suspicious in the metadata. The file appears to be a single long line of Base64-encoded data.

Step 5: Decode the Base64 Content

Since the file is Base64-encoded, decode it to reveal the hidden content:

```
cat logs.txt | base64 -d > decoded.png
```

This creates a PNG image file named `decoded.png`.

Step 6: Analyze the Decoded Image

Open the decoded image file. You may notice some hexadecimal text within it:

7069636F4354467B666F72656E736963735F616E616C797369735F69735F616D617A696E675F6563313938
3466637D

This appears to be hexadecimal encoding.

Step 7: Decode the Hexadecimal String

Convert the hexadecimal string to text using `xxd`:

```
echo
'7069636F4354467B666F72656E736963735F616E616C797369735F69735F616D617A696E675F656331393
83466637D' | xxd -r -p
```

✅ Final Result

```
picoCTF{<REDACTED>}
```

🧠 Key Concepts

- **Base64 Encoding**: A common method to encode binary data as text
- **Hexadecimal Encoding**: Representing binary data using base-16 digits
- **File Analysis**: Examining file types, metadata, and content for hidden information
- **Layered Encoding**: Multiple layers of encoding to conceal data